Credit Card Fraud Detection at Enterprise Scale

A First-Principles Machine Learning Evaluation, 100-Run Monte Carlo Benchmark, and Financial Decision Analysis in Pure NumPy

Course: Python for Data Science (PDS)	**Dataset:** Kaggle / ULB Benchmark (284,807 transactions)
Framework: 100% Pure NumPy (Zero External ML Packages)	**Class Imbalance:** 0.1727% (492 Frauds : 284,315 Genuine)
Evaluation Protocol: 100 Stratified Monte Carlo Splits (Seeds 1000–1099)	**Hardware Target:** 12-Core Multi-Threaded Local Benchmarking
Institution: IIIT Sri City — Capstone Repository	**Primary Methodology:** Asymmetric Cost-Sensitive Risk Minimization

1. Abstract & Research Objectives:
Credit card fraud detection operates under severe class imbalance and highly asymmetric financial consequences. In this investigation, we engineer and evaluate an end-to-end fraud screening pipeline built entirely from first principles in pure Python and vectorized NumPy, without reliance on high-level machine learning frameworks such as scikit-learn, XGBoost, or PyTorch. Our methodology encompasses ten modular techniques, including Synthetic Minority Oversampling (SMOTE), Deep Multi-Layer Perceptrons (MLP) trained via analytical backpropagation and Adam optimization, Gradient Boosted Decision Trees (GBDT), Unsupervised Isolation Forests, Focal Loss, Dynamic Bayesian Risk Thresholding, and Stacked Generalization.

To rigorously assess performance while accounting for sampling variance across data partitions, we execute a 100-run Monte Carlo benchmark across 100 pseudo-random seeds. Rather than viewing high recall in isolation, we formulate an asymmetric financial decision framework that balances direct fraud loss ($C_{\text{FN}} = \text{Amount}$) against false alarm investigation overhead (C_{FP}). Empirical findings demonstrate that while Cost-Sensitive Logistic Regression attains the highest event recall (93.81% ± 0.51%) and dollar recall (92.89% ± 1.41%), it generates an average of 7,854 false positives per test split, causing its total operational loss to exceed $20,500 under standard review costs. Conversely, GBDT and Baseline Logistic Regression provide substantially superior operational efficiency when false-positive handling costs are non-zero. Finally, we provide sensitivity analyses across cost assumptions, examine methodological limitations including concept drift, and outline a fully reproducible open-source implementation.

Formal Research Objectives:

1. **First-Principles Implementation:** Demonstrate that state-of-the-art fraud screening architectures can be implemented purely via vectorized numerical linear algebra without black-box machine learning libraries.

2. **Financial Decision Formulation:** Replace symmetric accuracy metrics with an asymmetric cost matrix reflecting direct stolen loss and investigation overhead.

3. **Uncertainty Quantification:** Evaluate estimator stability across 100 stratified partitions to report rigorous confidence intervals.

4. **Operational Trade-Off Analysis:** Quantify the tension between fraud capture and false-positive investigation capacity under varying operational cost regimes.

2. Dataset Characteristics and Preprocessing Pipeline

2.1 Data Provenance and Imbalance: The corpus comprises 284,807 credit card transactions recorded across European cardholders in September 2013, released by the Machine Learning Group at Université Libre de Bruxelles (ULB) and Worldline. The data contains 492 confirmed fraudulent events against 284,315 genuine transactions, yielding an extreme fraud incidence of **P(Y=1) = 0.1727%** (1 fraud per 578 legitimate transactions). A naive majority classifier predicting 'genuine' for all cases yields 99.827% accuracy yet captures zero fraud, demonstrating why accuracy is fundamentally uninformative.

2.2 Privacy Transformation (Features V1–V28): Due to confidentiality mandates (GDPR, PCI-DSS), original customer identifiers, merchant metadata, and locations were projected via Principal Component Analysis (PCA) into 28 orthogonal numerical features (V_1 to V_{28}). Mathematically, the centered feature matrix X was decomposed via Singular Value Decomposition: $X = U \Sigma V^T$, where the columns of V represent orthogonal eigenvectors of the covariance matrix. Consequently, $\text{Cov}(V_i, V_j) = 0$ for all $i \neq j$. Only `Time` (seconds elapsed from t_0) and `Amount` (transaction currency value) remain in their original scalar dimensions.

2.3 Missing Values and Outlier Audit: An exhaustive audit confirmed zero null, NaN, or missing values across all 284,807 rows. `Amount` exhibits extreme positive skewness (min: $0.00, median: $22.00, mean: $88.35, max: $25,691.16). To mitigate gradient explosion during numerical optimization, `Amount` is scaled using robust statistics (median and interquartile range) or standard normalization.

2.4 Strict Data Leakage Prevention Protocol: In highly imbalanced classification, data leakage between training and evaluation partitions is a frequent source of falsely inflated performance. We enforce an inviolable sequence of operations:

• **Step 1 (Partitioning):** The raw dataset is partitioned into an 80% training set (227,845 rows) and a 20% test set (56,962 rows) using stratified sampling to preserve the 0.1727% fraud ratio in both splits.

• **Step 2 (Parameter Estimation):** Scaling parameters (mean μ_{train}, standard deviation σ_{train}) are computed strictly on the training partition. Test features are standardized using these frozen training statistics.

• **Step 3 (Minority Oversampling):** Synthetic Minority Oversampling (SMOTE) is applied *exclusively* to the training partition. Synthetic instances are NEVER introduced into validation folds or the held-out test split, ensuring unbiased evaluation.

DATA PIPELINE & ISOLATION FLOW:

```
[Raw Data: 284,807 txns] ■■■ [Stratified 80/20 Split] ■■■■■■ [Train Split: 227,845 txns] ■■■ Fit Scaler (μ, σ) ■■■ Apply SMOTE ■■■ Train Model
■■■■ [Test Split: 56,962 txns] ■■■ Transform (Frozen μ, σ) ■■■■■■■■■■■■ Final Evaluation
```

3. Methodology and Model Implementation (First Principles)

All models and sampling routines were authored strictly from mathematical first principles in Python using vectorized NumPy. Below, each of the ten technical milestones is detailed with its objective, mathematical formulation, implementation, and known limitations:

Step 1: Synthetic Minority Oversampling (SMOTE) & Cyclic Time Encoding

Objective: Alleviate gradient starvation of minority class without simple instance duplication.
Mathematical Formulation: For each minority vector x_i, identify its k nearest minority neighbors under Euclidean distance. Synthesize a new sample: $x_{\text{syn}} = x_i + \lambda(x_{zi} - x_i)$ where $\lambda \sim \text{Uniform}(0, 1)$. Transform linear time t into circular topology: $\sin(2\pi t / 86400)$ and $\cos(2\pi t / 86400)$.
NumPy Implementation: Pairwise distance matrix computed via broadcasted differences: `d = np.linalg.norm(X[:, None] - X[None, :], axis=-1)`.
Limitations: Convex interpolation assumes local linearity between fraud points; can generate unrealistic samples in high-dimensional sparse voids.

Step 2: Deep Multi-Layer Perceptron (MLP) with Backpropagation and Adam

Objective: Model non-linear decision boundaries across orthogonal PCA features.
Mathematical Formulation: 3-layer architecture: $Z^{[l]} = W^{[l]}A^{[l-1]} + b^{[l]}$, with LeakyReLU activation $f(z) = \max(0.01z, z)$. Loss: Binary Cross-Entropy with L2 regularization $\frac{\lambda}{2N}\sum ||W||^2$. Gradients derived via analytical chain rule. Parameters updated via Adam: $m_t = \beta_1 m_{t-1} + (1-\beta_1)g_t$, $v_t = \beta_2 v_{t-1} + (1-\beta_2)g_t^2$ with bias correction.
NumPy Implementation: Fully vectorized forward and backward passes using matrix dot products (`@`) and He normal weight initialization.
Limitations: Computationally expensive relative to linear models; susceptible to uncalibrated probability predictions in tail distributions.

Step 3: Stratified K-Fold Cross-Validation & Asymmetric Cost Objective

Objective: Prevent fold collapse on rare classes and align validation with business costs.
Mathematical Formulation: Partition minority indices $\mathcal{I}_1$ and majority indices $\mathcal{I}_0$ into $K=5$ disjoint subsets, re-merging them to ensure exact class proportions. Cost Objective: $\mathcal{L}_{\text{cost}} = \sum (\text{FP} \times C_{\text{FP}}) + \sum (\text{FN} \times \text{Amount})$.
NumPy Implementation: Programmed custom stratified split generator using index slicing without external dependencies.
Limitations: 5-fold cross-validation scales training time 5×; metric variance can still remain non-trivial across folds.

Step 4: Gradient Boosted Decision Trees (GBDT) from Scratch

Objective: Construct an ensemble of weak decision stumps targeting non-linear feature interactions.
Mathematical Formulation: Minimize negative log-likelihood via functional gradient descent. At stage m, compute pseudo-residuals $r_{im} = y_i - p_i(x_i)$. Fit decision stump $h_m(x)$ by evaluating feature split thresholds that maximize variance reduction: $\Delta \text{Var} = \text{Var}(S) - \frac{|S_L|}{|S|}\text{Var}(S_L) - \frac{|S_R|}{|S|}\text{Var}(S_R)$. Update: $F_m(x) = F_{m-1}(x) + \eta h_m(x)$ with $\eta = 0.1$.
NumPy Implementation: Vectorized quantile scanning across feature vectors to find split candidates.
Limitations: Tree depth restricted to stumps/shallow trees to maintain reasonable training latency in pure NumPy without C++ tree walkers.

Step 5: Unsupervised Anomaly Detection: Isolation Forest from Scratch

Objective: Detect novel fraud patterns without reliance on ground-truth training labels.
Mathematical Formulation: Construct $T=100$ isolation trees using random feature selection and uniform random split thresholds. Let $h(x)$ denote path length to isolate sample x. The anomaly score is $s(x, n) = 2^{-E(h(x))/c(n)}$, where $c(n) = 2(\ln(n-1) + 0.5772156649) - \frac{2(n-1)}{n}$.
NumPy Implementation: Recursive tree construction stored as nested array indices; scoring vectorized across trees.
Limitations: Lower supervised precision (8.36%) as it flags generic distributional anomalies rather than fraud-specific behaviors.

Step 6: Cost-Sensitive Loss Reweighting & Dollar Value Recall

Objective: Direct optimization toward financial recovery rather than unweighted event detection.
Mathematical Formulation: Weight sample loss by transaction monetary exposure: $w_i = (\text{Amount}_i / \mu_{\text{Amount}})$ if $y_i=1$, else 1.0. Gradient: $\nabla_W = \frac{1}{N}X^T (W \odot (\hat{y} - y))$. Dollar Recall: $\frac{\sum_{TP} \text{Amount}}{\sum_{\text{All Fraud}} \text{Amount}}$.
NumPy Implementation: Element-wise weight array broadcasting during logistic loss and gradient computation.
Limitations: Induces aggressive positive classification for large transactions, substantially elevating false alarm rates.

Step 7: Multi-Paradigm Hybrid Soft-Voting Ensemble

Objective: Reduce model-specific variance by combining structurally diverse model families.
Mathematical Formulation: Calibrated probability blending: $P_{\text{ens}}(x) = 0.35 \cdot P_{\text{LogReg}}(x) + 0.35 \cdot P_{\text{GBDT}}(x) + 0.30 \cdot P_{\text{MLP}}(x)$.
NumPy Implementation: Linear combination of output probability vectors.
Limitations: Inference latency equals the sum of base models; requires probability calibration across disparate loss objectives.

Step 8: Dynamic Instance-Dependent Bayesian Thresholding

Objective: Replace static threshold ($p \ge 0.50$) with rational transaction-specific risk decisioning.
Mathematical Formulation: Under Bayes Minimum Risk, flag transaction x if expected cost of flagging is less than expected cost of allowing: $(1 - p(x))C_{\text{FP}} < p(x)C_{\text{FN}}(x) \implies p^*(x) = \frac{C_{\text{FP}}}{C_{\text{FP}} + \text{Amount}(x)}$.
NumPy Implementation: Vectorized element-wise comparison: `pred = (probs >= (2.50 / (2.50 + amounts))).astype(int)`.
Limitations: Assumes risk neutrality and known fixed C_{FP}; extreme low amounts ($0.99) require very high confidence to trigger.

Step 9: Focal Loss and Hard Negative Example Mining

Objective: Prevent overwhelmingly easy genuine transactions from dominating backpropagation gradients.
Mathematical Formulation: $\mathcal{L}_{\text{FL}}(p_t) = -\alpha_t (1 - p_t)^\gamma \ln(p_t)$, setting $\gamma = 2.0$. For well-classified genuine records ($p_t = 0.99$), modulating factor $(1 - 0.99)^2 = 0.0001$ suppresses gradient magnitude by $10,000\times$.
NumPy Implementation: Analytical focal loss gradient vectorization: $\nabla_W = \frac{1}{N}X^T ((1-p_t)^\gamma (p - y))$.
Limitations: Sensitive to hyperparameter selection (α, γ); can slow convergence in early training epochs.

Step 10: Stacked Generalization with Out-of-Fold Meta-Learning

Objective: Train a meta-learner to discover non-linear consensus among base predictors without data leakage.
Mathematical Formulation: Wolpert (1992) Stacking. Partition training data into 5 folds. Base models generate Out-Of-Fold (OOF) predictions matrix $S \in \mathbb{R}^{N \times M}$. A Level-1 Logistic Meta-Learner trains on S to learn model trust coefficients: $\hat{y} = \sigma(S W_{\text{meta}} + b_{\text{meta}})$.
NumPy Implementation: 5-fold cross-validation loop populating OOF matrix, followed by L2-regularized logistic regression on meta-features.
Limitations: High computational complexity during training; Level-1 learner can overfit if base models produce highly collinear probabilities.

4. Experimental Design and Benchmark Results

4.1 Simulation Design & Uncertainty Quantification: Evaluating an imbalanced fraud model on a single 80/20 train/test split is subject to sampling variance. To quantify estimator stability, we conducted a **100-run Monte Carlo simulation ($N = 100$)**. In each run $k \in \{1000, \dots, 1099\}$, the entire dataset was partitioned via a fresh stratified 80/20 split, all seven model architectures were trained from scratch, and test metrics were logged.

We report sample mean $\bar{x}$, sample standard deviation s, and the 95% Confidence Interval half-width for the mean: $\text{Margin} = 1.96 \cdot \frac{s}{\sqrt{N}} = 0.196 \cdot s$. *Note on Interpretation:* These confidence intervals quantify uncertainty in the estimated mean performance across repeated random splits of this historical dataset. They do NOT imply that sampling variance is eliminated, nor do they guarantee identical performance on future non-stationary data.

Model	Event Recall	Dollar Recall	Precision	F1 Score	ROC-AUC	FPR (%)	Saved ($)	False Pos (FP)	Missed ($)	Total Cost ($)
1. Baseline LogReg	86.74% ±0.66%	78.50% ±1.95%	14.29% ±0.34%	24.49% ±0.50%	0.9749 ±0.0019	0.91% ±0.03%	$9,508 ±$381	518.3 ±14.5	$2,672 ±$275	**$3,968 ±$275**
2. Cost-Sensitive LogReg	**93.81% ±0.51%**	**92.89% ±1.41%**	1.18% ±0.03%	2.33% ±0.06%	0.9732 ±0.0020	13.81% ±0.40%	**$11,254 ±$401**	7,854.0 ±225.6	**$926 ±$198**	$20,561 ±$508
3. Neural Net (MLP)	89.81% ±0.56%	84.35% ±1.84%	5.06% ±0.13%	9.57% ±0.24%	0.9716 ±0.0024	2.96% ±0.08%	$10,219 ±$394	1,682.8 ±47.6	$1,961 ±$269	$6,169 ±$270
4. GBDT Classifier	88.14% ±0.60%	81.67% ±1.86%	9.18% ±0.36%	16.57% ±0.58%	0.9771 ±0.0018	1.57% ±0.07%	$9,890 ±$382	892.7 ±39.5	$2,290 ±$272	**$4,522 ±$272**
5. Isolation Forest (Unsup.)	59.90% ±1.87%	56.13% ±2.86%	8.36% ±0.43%	14.59% ±0.66%	0.9449 ±0.0025	1.18% ±0.05%	$6,810 ±$431	673.0 ±30.5	$5,370 ±$409	$7,053 ±$410
6. Soft-Voting Ensemble	89.70% ±0.59%	84.51% ±1.75%	6.45% ±0.17%	12.01% ±0.28%	**0.9799 ±0.0016**	2.28% ±0.06%	$10,236 ±$388	1,299.3 ±36.7	$1,944 ±$240	$5,192 ±$240
7. Stacked Meta-Learner	90.34% ±0.59%	85.82% ±1.66%	4.57% ±0.11%	8.68% ±0.20%	0.9779 ±0.0019	3.31% ±0.09%	$10,399 ±$392	1,880.5 ±50.0	$1,781 ±$230	$6,482 ±$230

4.2 Critical Operational Trade-Off Analysis:

• **The False-Alarm Penalty:** Cost-Sensitive Logistic Regression attains the highest recall (93.81%), missing only 6.1 frauds ($926 loss). However, this comes at an extreme operational penalty: **7,854 false positives (13.81% FPR)**. Under a baseline review cost of $2.50 per alert, investigation overhead amounts to $19,635, elevating total operational loss to **$20,561 per split**.

• **Cost-Effective Champions:** Baseline Logistic Regression and GBDT exhibit far superior total cost profiles ($3,968 and $4,522 total loss, respectively). By restricting false alarms to 518 and 893, they prevent operational team saturation while preserving ~80% of stolen capital.

• **Reconciliation of Step 10 Notebook vs. 100-Run Table:** In the single-split Step 10 notebook (Seed 42), the stacked meta-learner was coupled with instance-dependent dynamic thresholding ($p^*(x)$), yielding 42.16% precision and 118 false alarms. In contrast, the 100-run simulation evaluated the stacked meta-learner under standard fixed thresholding ($p=0.50$) to ensure controlled benchmark parity across all models.

5. Financial Decision Analysis & Operational Cost Sensitivity

5.1 Cost Formulation & Institutional Grounding: A rational fraud system must minimize expected net loss. We define direct fraud loss as $C_{\text{FN}}(x) = \text{Amount}(x)$ due to mandatory consumer indemnification under US Regulation E / EU PSD2. The cost of a false alert (C_{FP}) encapsulates direct communication (automated SMS gateway $0.05, IVR phone call $0.20), amortized analyst triage queue time ($1.20–$1.80), and customer friction/churn hazard ($0.70–$1.20). While peer-reviewed literature (Bahnsen et al., 2015; Dal Pozzolo et al., 2014) identifies $C_{\text{FP}} \in [\$2.00, \$3.00]$ (standardized to $2.50), we treat $C_{\text{FP}} = \$2.50$ as a **project baseline assumption** rather than a universal constant.

5.2 Bayes Minimum Risk Threshold Derivation: Under risk-neutral expectation, an alert is triggered if: $(1 - p(x))C_{\text{FP}} < p(x)C_{\text{FN}}(x) \implies p^*(x) = \frac{C_{\text{FP}}}{C_{\text{FP}} + \text{Amount}(x)}$. This dynamic rule scales threshold sensitivity: high-dollar transactions ($3,000) trigger at $p^* \approx 0.083\%$, while micro-transactions ($0.99) require $p^* \approx 71.6\%$, preventing negative expected value interventions.

5.3 Empirical Sensitivity Analysis Across Cost Regimes: Because actual operational overhead varies widely across institutions (e.g., automated in-app biometric challenge vs. manual phone review), we evaluated the total expected financial loss $\mathbb{E}[\text{Loss}] = \text{FP} \times C_{\text{FP}} + \text{Missed Dollars}$ across 100 runs for $C_{\text{FP}} \in [\$0.50, \$1.00, \$2.50, \$5.00, \$10.00]$:

Model Architecture	C_FP = $0.50 (Push/App)	C_FP = $1.00 (SMS Alert)	C_FP = $2.50 (Base/Queue)	C_FP = $5.00 (Manual Call)	C_FP = $10.00 (Analyst Review)	Optimal Regime
1. Baseline Logistic Regression	$2,931.06	$3,190.22	**$3,967.70**	**$5,263.50**	**$7,855.10**	**Lowest Cost for C_FP ≥ $2.50**
2. Cost-Sensitive LogReg	$4,853.38	$8,780.38	$20,561.36	$40,196.34	$79,466.29	Viable only if C_FP < $0.15
3. Neural Network (MLP)	$2,802.91	$3,644.33	$6,168.61	$10,375.73	$18,789.98	Balanced Non-Linear Catch
4. GBDT Classifier	$2,736.38	**$3,182.72**	$4,521.74	$6,753.44	$11,216.84	**Lowest Cost for C_FP ≈ $1.00**
5. Isolation Forest (Unsup.)	$5,706.44	$6,042.97	$7,052.54	$8,735.17	$12,100.42	Zero-Day Anomaly Sentinel
6. Soft-Voting Ensemble	**$2,593.39**	$3,243.02	$5,191.91	$8,440.06	$14,936.36	**Lowest Cost for C_FP ≤ $0.50**
7. Stacked Meta-Learner	$2,721.38	$3,661.66	$6,482.48	$11,183.86	$20,586.61	High Precision Meta-Ranking

Key Sensitivity Finding: If false alarms can be verified via low-friction, automated digital channels ($C_{\text{FP}} \le \$0.50$), the **Soft-Voting Ensemble** minimizes total institutional loss ($2,593.39). As investigation overhead rises to standard operational levels ($C_{\text{FP}} \ge \$2.50$), **Baseline Logistic Regression** emerges as the most financially efficient model ($3,967.70), followed closely by GBDT ($4,521.74). Cost-Sensitive Logistic Regression is economically unviable unless C_{FP} approaches zero.

6. Discussion, Limitations, and Practical Recommendations

6.1 Synthesis of Experimental Findings:

1. *The Trade-Off Between Detection and Operational Overhead:* Maximizing recall does not equate to minimizing cost. High-recall models (Cost-Sensitive LogReg) generate massive alert volumes that can overwhelm operational fraud investigation queues.
2. *First-Principles Vectorization:* By structuring forward propagation, gradient descent, and decision tree splitting into vectorized NumPy array operations, we attained sub-millisecond per-sample scoring latency on standard hardware without relying on compiled C++ frameworks.
3. *Unsupervised Complements Supervised:* While Isolation Forest achieved lower overall recall (59.90%), its zero-label formulation renders it an essential parallel watchdog for detecting structural anomalies that evade supervised boundary planes.

6.2 Operational Model Deployment Selection Guide:

• **Scenario A (Strict Alert Capacity / Manual Triage):** Deploy **Baseline Logistic Regression** or **GBDT**. These models maintain FPR < 1.6%, keeping weekly false positives within manageable analyst limits while stopping > 86% of fraud.
• **Scenario B (Automated Frictionless Customer Challenge):** Deploy the **Soft-Voting Ensemble**. It delivers the highest ROC-AUC (0.9799) and minimizes total cost when automated verification costs are low ($C_{\text{FP}} \le \$0.50$).
• **Scenario C (Catastrophic High-Value Protection):** Deploy **Cost-Sensitive LogReg** strictly as a secondary filter on high-dollar transactions (> $1,000), where saving $4,000+ per event economically justifies false positive inquiries.

6.3 Study Limitations & Methodological Caveats:

• *Non-Stationary Adversarial Concept Drift:* This dataset reflects a static 48-hour snapshot from 2013. In production, fraud syndicates actively probe rules, adapt merchant categories, and rotate card vectors. A static model trained on historical data will experience performance decay.
• *Synthetically Derived Features:* PCA anonymization obscured semantic transaction attributes (merchant terminal ID, device fingerprint, IP velocity), preventing real-time graph relational modeling.
• *Finite Pool Resampling:* While 100 Monte Carlo splits evaluate algorithmic variance under repeated partitioning, all splits draw from the same finite pool of 492 fraud events. True generalization requires streaming out-of-time evaluation across distinct temporal epochs.

6.4 Proposed Latency Design Targets vs. Production Reality:

Commercial networks (VisaNet, Mastercard Banknet) mandate strict transaction scoring budgets (< 50 ms). In our local test environment, single-sample inference averaged 0.2 ms for Logistic Regression, 1.8 ms for MLP, and 4.5 ms for GBDT. However, these figures represent **proposed engineering targets on benchmark hardware**, not field-validated production SLAs under concurrent multi-tenant network loads.

7. References and Reproducibility Appendix

7.1 Traceable Peer-Reviewed Bibliography:

• Bahnsen, A. C., Aouada, D., Stojanovic, A., & Ottersten, B. (2015). 'Cost-sensitive credit card fraud detection using Bayes minimum risk.' *Expert Systems with Applications*, 42(5), 2501–2509. DOI: 10.1016/j.eswa.2014.10.038.
• Dal Pozzolo, A., Caelen, O., Le Borgne, Y. A., Waterschoot, S., & Bontempi, G. (2014). 'Learned lessons in credit card fraud detection from a practitioner perspective.' *Expert Systems with Applications*, 41(10), 4915–4928. DOI: 10.1016/j.eswa.2014.02.026.
• Dal Pozzolo, A., Borne, Y. A., Caelen, O., & Bontempi, G. (2015). 'Calibrating probability with undersampling for unbalanced classification.' *IEEE Symposium Series on Computational Intelligence (SSCI)*, 159–166. DOI: 10.1109/SSCI.2015.33.
• Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). 'SMOTE: synthetic minority over-sampling technique.' *Journal of Artificial Intelligence Research (JAIR)*, 16, 321–357. DOI: 10.1613/jair.953.
• Wolpert, D. H. (1992). 'Stacked generalization.' *Neural Networks*, 5(2), 241–259. DOI: 10.1016/S0893-6080(05)80023-1.
• Friedman, J. H. (2001). 'Greedy function approximation: a gradient boosting machine.' *The Annals of Statistics*, 29(5), 1189–1232. DOI: 10.1214/aos/1013203451.
• Liu, F. T., Ting, K. M., & Zhou, Z. H. (2008). 'Isolation forest.' *IEEE International Conference on Data Mining (ICDM)*, 413–422. DOI: 10.1109/ICDM.2008.17.
• Lin, T. Y., Goyal, P., Girshick, R., He, K., & Dollár, P. (2017). 'Focal loss for dense object detection.' *IEEE International Conference on Computer Vision (ICCV)*, 2980–2988. DOI: 10.1109/ICCV.2017.324.
• Davis, J., & Goadrich, M. (2006). 'The relationship between Precision-Recall and ROC curves.' *International Conference on Machine Learning (ICML)*, 233–240. DOI: 10.1145/1143844.1143874.
• LexisNexis Risk Solutions. (2023). '2023 True Cost of Fraud Study: Financial Services and Lending.' Global Financial Fraud Report.
• United States Code of Federal Regulations. 12 CFR Part 1005 (Regulation E) — Electronic Fund Transfers. Consumer Financial Protection Bureau (CFPB).

7.2 Environment and Execution Instructions:

• **Hardware / Runtime:** Python 3.13 / Windows 11 x64, 12 Logical CPU Cores, 16 GB RAM.
• **Dependencies:** `numpy>=1.24.0`, `pandas>=2.0.0`, `reportlab>=4.0.0` (ReportLab used strictly for PDF rendering; zero external ML packages).
• **Re-generating 100-Run Benchmark:** Run `python run_100_simulations.py` (executes 100 parallel worker processes across seeds 1000–1099 in ~88s).
• **Re-generating Monograph PDF:** Run `python generate_pdf_report.py` to re-compile this PDF to both the workspace and Desktop.